

Reviewer Pack — Real Rank of Coefficient Matrices for AC^0 PARITY Circuits

Entry 80c8560ef86e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 06:41:53 UTC

Real Rank of Coefficient Matrices for AC^0 PARITY Circuits

Entry ID: 80c8560ef86e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 06:41:53 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC^0 Circuits Computing PARITY

Statement:

For any AC^0 circuit C computing PARITY on n inputs, the real rank of the matrix formed by the coefficients of the linear terms in C is $\Omega(\log \text{size}(C))$

Rationale (proposer’s reasoning):

The real rank captures geometric constraints on linear representations, and AC^0 circuits must exhibit high-rank coefficient matrices to approximate PARITY, which is inherently linear.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 132f8bda10b5e4df

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    n = 40
    depth = 5
    size_C = 2**n * depth

    # Generate a random AC0 circuit for PARITY on n inputs
    def generate_ac0_circuit(n, depth):
        circuit = []
        for _ in range(depth):
            layer = [random.choice(['AND', 'OR']) if i % 2 == 1 else 'NOT' for i in range(2**(n-1))]
            circuit.append(layer)
        return circuit

    circuit = generate_ac0_circuit(n, depth)

    # Construct the coefficient matrix
    def construct_coefficient_matrix(circuit, n):
        num_inputs = 2**n
        coefficient_matrix = [[0] * num_inputs for _ in range(num_inputs)]

        def evaluate_expression(expr, inputs):
            if expr == 'NOT':
                return not inputs[0]
            elif expr == 'AND':
                return all(inputs)
            elif expr == 'OR':
                return any(inputs)

            left, right = expr.split(' ')
            left_val = evaluate_expression(left, inputs[:len(inputs)//2])
            right_val = evaluate_expression(right, inputs[len(inputs)//2:])
            if expr.startswith('NOT'):
                return not (left_val and right_val)
            elif expr.startswith('AND'):
                return left_val and right_val
            elif expr.startswith('OR'):
                return left_val or right_val

        for i in range(num_inputs):
            for j in range(num_inputs):
                inputs = [bool(i & (1 << k)) for k in range(n)]
```

```

        result = evaluate_expression(circuit[0][i % len(circuit[0])], inputs)
        coefficient_matrix[i][j] = Fraction(result, 2**n)

    return coefficient_matrix

coefficient_matrix = construct_coefficient_matrix(circuit, n)

# Compute the real rank via eigenvalue decomposition
def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(cols):
            if i != j:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(rows):
                    matrix[j][k] -= factor * matrix[i][k]

    rank = sum(1 for row in matrix if any(val != 0 for val in row))
    return rank

real_rank = gaussian_elimination(coefficient_matrix)

# Verify the conjecture
min_rank = Fraction(1, 10) * math.log(size_C, 2)
conjecture_holds = real_rank >= min_rank
counterexample = "" if conjecture_holds else "rank < 0.1 * log(size(C))"

return {
    "metric_name": "real_rank",
    "metric_value": real_rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        random.seed(seed)
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(res["metric_value"] for res in results) / len(results)
    std_rank = math.sqrt(sum((res["metric_value"] - mean_rank)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])

```

```

print(f"RESULT: FALSIFIED counterexample=\"rank < 0.1 * log(size(C))\" first_failing_seed=
else:
print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with identical parameters

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	53591
2	preregistration	ollama_remote	qwen3:8b	0	0	24132
3	novelty	ollama_remote	qwen3:8b	0	0	20811
4	novelty	ollama_remote	qwen3:8b	0	0	14342
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13575
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11137
7	judge	ollama_remote	qwen3:8b	0	0	37060

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 174647 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/80c8560ef86e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/80c8560ef86e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/80c8560ef86e.tar.gz (if generated)